

Extension of Understanding of Raft Consensus

Ovidiu Marcu

Election Safety: at most one leader can be elected in a given term. §5.2

Leader Append-Only: a leader never overwrites or deletes entries in its log; it only appends new entries. §5.3

Log Matching: if two logs contain an entry with the same index and term, then the logs are identical in all entries up through the given index. §5.3

Leader Completeness: if a log entry is committed in a given term, then that entry will be present in the logs of the leaders for all higher-numbered terms. §5.4

*Our clarification: “a given term” must be read as the commit term — the current term of the leader whose commit made the entry durable — not the term stored in the entry itself (its authoring term). The two coincide for directly committed entries and differ for entries committed indirectly (the Figure 8 scenario), where leaders of the terms in between are legitimately exempt. The TLA⁺ definitions of both readings, together with the commit-certificate fix, are given in [1]; the commit-term reading is carried by invariant #4 (LeaderCompleteness, keyed on the recorded certificate term *c.term*).*

Our Leader Completeness: if a log entry becomes committed while its leader is at term *T* (the *commit* term, recorded at the moment of commitment), then that entry will be present in the logs of the leaders for all terms greater than *T*. §5.4

State Machine Safety: if a server has applied a log entry at a given index to its state machine, no other server will ever apply a different log entry for the same index. §5.4.3

Ack Retention: if a server’s acknowledgment of a log entry has been counted toward committing that entry, then the server still stores that entry whenever it subsequently grants a vote (a guarantee about what is true at every vote-grant, not a condition the voter checks). §5.4.3 (step 4)

Commit in Raft is a leader-side act: on seeing an entry of its own current term acknowledged by a majority, the leader advances its `commitIndex` — nothing else happens. From that instant the entry is durable and every future leader must contain it. Every entry thus has two terms. Its authoring term is the number written inside the entry at creation — the term of the leader that first appended it — and it never changes. Its commit term is the current term of the leader whose `commitIndex` advance made it durable. Example: a term-2 leader creates an entry (authoring term 2) but crashes before committing it; a term-4 leader inherits the entry, appends its own term-4 entry above it, replicates both to a majority, and advances `commitIndex` over both — the inherited entry still carries the number 2 inside, yet it became committed at term 4: authoring term 2, commit term 4. For an entry the committing leader created itself, the two coincide.

“4. The voter still stored the entry when it voted for leaderU, since every intervening leader contained the entry (by assumption), leaders never remove entries, and followers only remove entries if they conflict with the leader.”

The sixth guarantee is derived from one clause of the §5.4.3 proof sketch, quoted verbatim above — yet the five listed properties lean on it: State Machine Safety reduces to Leader Completeness, whose proof depends, at the quorum-intersection voter, on exactly this retention. Ack Retention is why the guarantees compose.

Figure 3 (extended): Raft guarantees that each of these properties is true at all times. The section numbers indicate where each property is discussed. The first five entries are Figure 3 of Ongaro & Ousterhout [2]; the clarification notes and the sixth entry are additions of this work.

References

- [1] Ovidiu Marcu et al. *From Ambiguous Prose to a Verified Invariant: A Leader-Completeness Bug in the CCF TLA⁺ Specification and Its Commit-Certificate Fix*. <https://github.com/granular-storage/RaftFix/blob/main/paper.pdf>
- [2] Diego Ongaro and John Ousterhout. *In Search of an Understandable Consensus Algorithm (Extended Version)*. <https://raft.github.io/raft.pdf>